

## 1. Tell a story about your design

### a. What optimizations did you try? Which ones worked? Which ones didn't work?

Critical path optimization

TAGE, Gshare and Local branch predictors

Writeback Cache (didn't work too well)

Out-of-order to reduce RAW stalling penalty (did not work)

Forwarding data from MEM2 to ID

### b. For each optimization, discuss in detail the effects on your critical path, IPC, cache hit rate, power, area, and other relevant design metrics... How did you know that an optimization was or was not "worth it"?

We optimized our Lab 4A critical path significantly, cutting the clock period down from 4.4ns to 3.28ns, which represents a 36% frequency speedup. We pipelined the flush signal so that the critical path through the instruction cache would be eliminated (read\_instr, cancel\_instr etc.). This reduced our clock period down to 3.8ns.

We also made a major architectural change to reduce critical path down even further, by forwarding to decode instead of execute. This unfortunately resulted in an IPC tradeoff, since we are now unable to forward distance 3 loads (used to be able to forward WB->EX, but now we can only forward up till WB->ID). With this new clock period of 3.28ns, most of our critical paths were through execute and the mispredict logic. However, there were also a few other critical paths in top\_paths.rpt, such as the cache combinational logic and arbitration to memory. These would be difficult to eliminate without a major redesign of the cache controller / cache.

We also realized that the two-cycle latency of the cache\_controller was longer than necessary, and did not need the FIFO queue. We could achieve a 1-cycle latency by taking the data from the cache, bypassing the FIFO and passing that data to the instr in the MEM2 stage. That would then be forwarded to ID, eliminating stalls due to dist 3 loads.

### c. What was your design process? We're not paying attention to the steps to achieve correctness, rather the ones that you took between the minimal solution and the one that you submitted. What decisions did you make after the design was correct to make it more efficient, correctly sized, etc.?

We decided from the Lab 4A cache parameter sweeps that the optimal cache parameter sizes for fft.c and kosarajus.c were 2 way, 6 index bits and LRU policy for the instruction cache and 2 way, 4 index bits and LRU policy for the data cache. Switching to 4-way for either the instruction / data cache only gave marginal improvements for kosarajus.c (on the order of 10000 cycles,

which is less than 1% of IPC improvement). For `fft.c`, a 4-way cache gave greater improvement in IPC (on the order of 700000 cycles for the instruction cache, or a 9% IPC improvement). Overall, we felt that the IPC improvement from a 4-way cache does not warrant the massive power consumption (which was 60mW higher than a 2-way cache).

For branch prediction, we made a number of relatively simple predictors, and some larger predictors that were less optimal. In testing, we found that the implementation overhead of TAGE and gshare was too much for a relatively small payoff. Local predictors were far better as training time was lower which was particularly suitable for high mispredict programs like `fft.c`. Overall, we ended up with a relatively small version of local that worked particularly well on `fft` while maintaining relatively low (<60k) mispredicts on `kosarajus`.

## **2. Convince us that you put effort into your design**

**a. If you spent a lot of time on an optimization that didn't work, we're still curious to learn about it and may assign credit for the effort you put into it.**

With more RAW load hazards due to forwarding into decode, we attempted an unconventional solution. We attempted to 'speedup' the load through the pipeline to reduce the number of stall cycles (if a consumer instruction was dependent on it). The goal was to have a distance 3 load be pushed a cycle faster through the pipeline, so that the distance increases to 4, eliminating stalls. This involved the load in EX stage sending a memory request in the same cycle, getting latched into MEM2, overtaking / bypassing the instruction in MEM1.

This unfortunately resulted in functional errors due to WAW hazards (since the older instruction in MEM1 will commit later than the younger load, committing the wrong state). We managed to get this implementation passing some of the benchmarks (gaussian, quicksort, transpose). For `gaussian.c`, we managed to shave down the cycle count from ~73500 to ~69500, which represents a 4000 cycle reduction due to less RAW hazard stalls, or a 6% improvement in IPC. We did not have a functional implementation to test on the perf benchmarks, and later realized that we could cut cache latency down to 1 cycle, solving this problem without an out-of-order implementation.

Our writeback cache did not work too well, and we had issues integrating it with our pipeline. We also realized that even if functional, the performance improvement due to a writeback cache was minimal. A writeback data cache would only help prevent memory accesses in the case of a store hit, reducing memory conflicts with the instruction cache. However, sweeps showed that I/D conflicts were at most 30,000 for the perf benchmarks, which was not a significant improvement (we could at most shave down 30,000 cycles out of millions), so we did not pursue this further.